

CM1 : Introduction à Python scientifique

Fondamentaux, NumPy, Matplotlib et méthodes numériques

Felipe FIGUEREDO-ROCHA (felipe.figueredo-rocha@u-pec.fr)

Méthodes de résolution numérique de problèmes d'analyse (Master 1 Mécanique UPEC-UGE)

22/09/26

- 30h (12h CM / 18h TP) : TP GPA (Sara TOUHAMI), TP GPB (Felipe ROCHA)
- Évaluation :
 - Contrôle Continu (50%) : 2 CRs de TP et soutenance orale.
 - Examen (50%) : Épreuve écrite individuelle en fin de semestre.
- Programme (division approximatif):
 - Python pour le calcul scientifique (CM1, TP1)
 - Fondements de l'analyse numérique (CM1, CM2, TP1)
 - Résolution d'équations non-linéaires (CM1, TP1, TP2)
 - Approximation et Interpolation (CM2, TP2, TP3)
 - Intégration Numérique (CM3, TP4)
 - Solution numérique des équations différentielles ordinaires (CM4, TP5, TP6).
- Références : Notes de cours et TPs des années précédents (github.com/felipefr/numerical_analysis), slides.

Objectifs du cours

À la fin de ce cours, vous serez capables de :

- écrire et exécuter des programmes Python simples ;
- manipuler des listes, tableaux et matrices ;
- utiliser NumPy pour le calcul scientifique ;
- tracer des courbes avec Matplotlib ;
- programmer une méthode numérique simple ;
- comparer Python à MATLAB et Fortran.

Fil conducteur

Résoudre numériquement une équation

$$f(x) = 0$$

à l'aide de la méthode de bisection.

Pourquoi Python ?

- Langage généraliste, libre et multiplateforme.
- Syntaxe lisible et relativement concise.
- Écosystème scientifique très riche :
 - NumPy : tableaux et algèbre numérique ;
 - SciPy : méthodes scientifiques avancées ;
 - Matplotlib : visualisation ;
 - Jupyter : calcul interactif.
 - Spyder : IDE à la Matlab.
 - VSCode : IDE professionnelle multilangage.
- Utilisé en recherche, ingénierie, data science et enseignement.

Exécuter du Python

Trois possibilités courantes :

① Interpréteur interactif :

```
>>> 2 + 3  
5
```

② Fichier .py :

```
python mon_programme.py
```

③ Notebook Jupyter .ipynb :

- cellules de code ;
- texte et formules ;
- graphiques intégrés ;
- exploration interactive.

Variables et types

```
x = 3
y = 2.5
nom = "Python"
actif = True

print(x)
print(type(y))
```

Types usuels :

- int : entier ;
- float : nombre réel approché ;
- complex : nombre complexe ;
- str : chaîne de caractères ;
- bool : booléen.

Attention

Python distingue l'affectation = de la comparaison ==.

Opérateur	Signification	Exemple
+, -	Addition, soustraction	$x + y$
*, /	Produit, division	x / y
**	Puissance	$x**2$
//	Division entière	$7//2$
%	Reste	$7\%2$
==, !=	Comparaison	$x == y$
<, >	Inégalités	$x < y$

Conditions

```
x = 2.5

if x > 0:
    print("x est positif")
elif x == 0:
    print("x est nul")
else:
    print("x est negatif")
```

Point important

L'indentation fait partie de la syntaxe de Python.

- Pas de end obligatoire.
- Les blocs sont définis par l'indentation.
- Utiliser généralement quatre espaces.

Boucle for

```
for k in range(5):  
    print(k)
```

- `range(5)` produit les entiers de 0 à 4.
- Une boucle `while` doit progresser vers une condition d'arrêt.
- Attention aux boucles infinies.

Boucle while

```
k = 0  
while k < 5:  
    print(k)  
    k += 1
```

Commandes break et continue

Boucle for

```
for i in range(10):  
    if i == 5:  
        break  
    print(i)
```

Boucle while

```
for i in range(6):  
    if i == 3:  
        continue  
    print(i)
```

- Quels sorties ?

Listes et indexation

```
a = [10, 20, 30, 40]

print(a[0]) # 10
print(a[-1]) # 40
print(a[1:3]) # [20, 30]

a.append(50)
a[0] = 100
```

Indexation

En Python, l'indexation commence à zéro.

- Une liste peut contenir des types différents.
- Les listes sont pratiques, mais moins adaptées que NumPy pour le calcul vectoriel intensif.

```
def moyenne(a, b):  
    return (a + b) / 2  
  
carre = lambda x: x**2  
  
print(carre(3))  
print(moyenne(2, 6))
```

Une fonction permet de :

- factoriser le code ;
- améliorer la lisibilité ;
- tester séparément une partie du programme ;
- réutiliser une méthode numérique.

```
import math

print(math.sqrt(2))
print(math.sin(math.pi / 2))
```

Autres formes :

```
from math import sqrt
from math import pi as PI
```

Bonne pratique

Importer explicitement les bibliothèques utilisées et éviter les imports inutiles.

Pourquoi NumPy ?

NumPy signifie *Numerical Python*.

Il fournit :

- des tableaux multidimensionnels : `ndarray` ;
- des opérations vectorisées ;
- des fonctions mathématiques ;
- des outils d'algèbre linéaire ;
- une représentation efficace des données numériques.

```
import numpy as np # convention forte
```

Créer des tableaux NumPy

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([[1, 2], [3, 4]])

z = np.zeros(5)
o = np.ones((2, 3))
I = np.eye(3)

x = np.arange(0, 1, 0.2) # array([0. , 0.2, 0.4, 0.6, 0.8])
y = np.linspace(0, 1, 6) # array([0. , 0.2, 0.4, 0.6, 0.8, 1. ])
```

- zeros : tableau de zéros ;
- ones : tableau de uns ;
- eye : matrice identité ;
- arange : pas fixé ;
- linspace : nombre de points fixé.

Forme, taille et type

```
A = np.array([[1, 2, 3],  
              [4, 5, 6]])  
  
print(A.shape) # (2, 3)  
print(A.ndim) # 2  
print(A.size) # 6  
print(A.dtype) # type des elements  
  
B = A.reshape((3,2)) # ou (3,-1), ou (-1,2)  
c = A.flatten()
```

Vocabulaire

- `shape` : dimensions du tableau ;
- `ndim` : nombre de dimensions ;
- `size` : nombre total d'éléments ;
- `dtype` : type numérique stocké.

Opérations vectorisées

```
x = np.array([1, 2, 3, 4])  
y = 2*x + 1  
z = x**2  
w = np.sin(x)
```

Sans boucle explicite, NumPy applique les opérations élément par élément.

Différence importante

* réalise un produit élément par élément. Pour un produit matriciel, on utilise @.

Opérations matricielles

```
A = np.array([[1, 2], [3, 4]])  
B = np.array([[5, 6], [7, 8]])  
  
C1 = A * B  
C2 = A @ B  
AT = A.T
```

- $A * B$: produit élément par élément ;
- $A @ B$: produit matriciel ;
- $A.T$: transposée ;
- `np.linalg.det(A)` : déterminant ;
- `np.linalg.inv(A)` : inverse, si elle existe.

Résoudre un système linéaire

On souhaite résoudre :

$$Ax = b.$$

```
A = np.array([[3, 1],  
              [1, 2]], dtype=float)  
  
b = np.array([9, 8])  
  
x = np.linalg.solve(A, b)  
print(x)
```

Bonne pratique

Préférer `np.linalg.solve(A,b)` au calcul explicite de `np.linalg.inv(A) @ b`.

Fonctions usuelles de NumPy

```
x = np.array([1, 2, 3, 4])
```

```
np.sum(x)
```

```
np.mean(x)
```

```
np.min(x)
```

```
np.max(x)
```

```
x[np.argmax(x)]
```

```
np.linalg.norm(x)
```

```
x[np.where(x>2)]
```

Autres fonctions utiles :

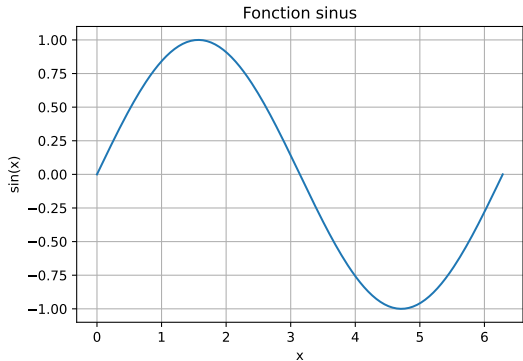
- `np.abs`, `np.sqrt`, `np.exp` ;
- `np.sin`, `np.cos`, `np.log` ;
- `np.argmax`, `np.argmin` ;
- `np.where` pour sélectionner selon une condition.

Visualiser avec Matplotlib

```
import numpy as np
import matplotlib.pyplot as plt

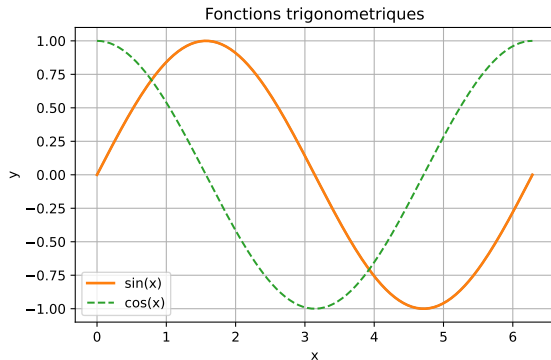
x = np.linspace(0, 2*np.pi, 200)
y = np.sin(x)

plt.plot(x, y)
plt.xlabel("x")
plt.ylabel("sin(x)")
plt.title("Fonction sinus")
plt.grid(True)
plt.show()
```



Personnaliser une figure

```
plt.plot(x, np.sin(x),  
         label="sin(x)", linewidth=2)  
  
plt.plot(x, np.cos(x),  
         label="cos(x)", linestyle="--")  
  
plt.xlabel("x")  
plt.ylabel("y")  
plt.title("Fonctions trigonometriques")  
plt.legend()  
plt.grid(True)  
plt.tight_layout()  
plt.show()
```



Sous-figures (Méthode 1)

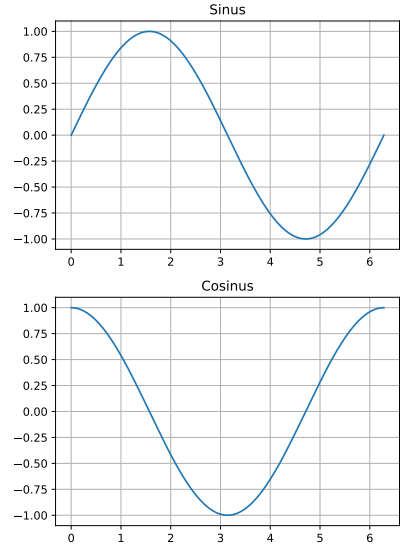
```
fig, ax = plt.subplots(1, 2, figsize=(9, 3))

ax[0].plot(x, np.sin(x))
ax[0].set_title("Sinus")

ax[1].plot(x, np.cos(x))
ax[1].set_title("Cosinus")

for a in ax:
    a.grid(True)

plt.tight_layout()
plt.show()
```



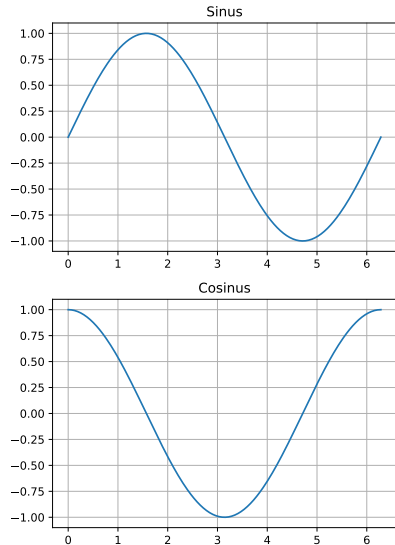
Sous-figures (Méthode 2)

```
plt.figure(5, figsize=(5, 7) )
```

```
plt.subplot(2, 1, 1)  
plt.title("Cosinus")  
plt.plot(x, np.cos(x))  
plt.grid()
```

```
plt.subplot(2, 1, 2)  
plt.title("Sinus")  
plt.plot(x, np.sin(x))  
plt.grid()
```

```
plt.tight_layout()  
plt.show()
```



Matplotlib permet notamment de tracer :

- courbes : `plot` ;
- nuages de points : `scatter` ;
- histogrammes : `hist` ;
- courbes de niveau : `contour` ;
- images et cartes de chaleur : `imshow`.

```
plt.scatter(x, y)  
plt.hist(data, bins=20)
```

Pourquoi utiliser des classes ?

Une classe permet de regrouper :

- des **données** : les attributs ;
- des **fonctions** : les méthodes ;
- des opérations associées à un même objet.

Programmation orientée objet

Un objet est une instance d'une classe. Il possède un état (attributs) et des comportements (méthodes).

```
class Particle:  
    pass  
  
p = Particle()
```

- Particle : la classe ;
- p : un objet de cette classe.

Attributs et méthodes

```
class Particle:

    def __init__(self, mass, position):
        self.mass = mass
        self.position = position

    def describe(self):
        print("Masse :", self.mass)
        print("Position :", self.position)

    def move(self, displacement):
        self.position += displacement
```

- `__init__` : initialise un nouvel objet ;
- `self` : référence vers l'objet courant ;
- `self.mass` et `self.position` : attributs ;
- `describe` et `move` : méthodes.

Créer et manipuler des objets

```
p1 = Particle(2.0, np.array([0.0, 0.0, 0.0]))
p2 = Particle(1.5, np.array([1.0, 2.0, 3.0]))

p1.describe()
p1.move(np.ones(3))

print(p1.position)
print(p2.position)
```

Concept important

Chaque objet possède ses propres attributs. Une méthode peut lire ou modifier l'état de l'objet via `self`.

- La classe définit une structure commune.
- Les instances peuvent avoir des valeurs différentes.
- Les méthodes regroupent les opérations liées à l'objet.

Exemple scientifique : une classe pour un maillage

Une classe peut regrouper les données et opérations d'un maillage numérique.

```
import numpy as np

class Mesh1D:

    def __init__(self, a, b, n):
        self.x = np.linspace(a, b, n+1)
        self.n = n

    def step(self):
        return self.x[1] - self.x[0]

    def display(self):
        print("Nombre de cellules :", self.n)
        print("Pas :", self.step())
```

- Les nœuds du maillage sont stockés dans un tableau NumPy.
- La méthode `step()` calcule le pas spatial.
- La classe peut être enrichie pour les méthodes numériques.

Héritage

L'**héritage** permet de créer une nouvelle classe à partir d'une classe existante.

- La classe dérivée réutilise les attributs et méthodes de la classe parente.
- Elle peut ajouter de nouvelles fonctionnalités.
- Elle peut redéfinir certaines méthodes.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        pass # not implemented

a = Animal('Newton')
a.speak()
```

```
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed

    def speak(self):
        print("Wouf !")

d = Dog('Einstein', 'German Shepherd')
d.speak()
```

Vocabulaire

Animal : classe parente (et dans ce cas une classe abstraite) Dog : classe dérivée

Langage	Caractéristiques
Python	Généraliste, très lisible, vaste écosystème scientifique.
MATLAB	Environnement intégré orienté calcul matriciel et ingénierie.
Fortran	Langage historique du calcul scientifique, performant pour certains calculs numériques.

Comparaison des styles

Aspect	Différence principale
Lisibilité	Python privilégie une syntaxe concise et l'indentation.
Matrices	MATLAB possède une syntaxe matricielle intégrée ; NumPy fournit cette fonctionnalité à Python.
Performance	Fortran peut être très performant pour des noyaux numériques compilés.
Écosystème	Python permet de combiner calcul, visualisation, données et automatisation.
Développement	Python facilite souvent le prototypage rapide.
Coût	Python et Fortran sont gratuits, MATLAB payant.

Nuance

La performance dépend de l'algorithme, des bibliothèques utilisées, de la compilation et de la taille des problèmes.

Python vs Fortran : comparaison de syntaxe

Python

```
# Variables
x = 2.0
n = 10

# Boucle
s = 0.0
for i in range(1, n+1):
    s += i**2

# Fonction
def square(x):
    return x**2

print(square(3.0))
```

Fortran

```
! Variables
real(8) :: x, s
integer :: n, i
x = 2.0d0
n = 10

! Boucle
s = 0.0d0
do i = 1, n
    s = s + i**2
end do

! Fonction
function square(x) result(y)
    real(8), intent(in) :: x
    real(8) :: y
    y = x**2
end function square
```

Python vs Fortran : tableaux

Python + NumPy

```
import numpy as np

x = np.array([1., 2., 3.])
y = 2*x + 1

A = np.array([[1., 2.],
              [3., 4.]])

b = A @ x[:2]
```

Fortran

```
real(8) :: x(3), y(3)
real(8) :: A(2,2), b(2)

x = [1.0d0, 2.0d0, 3.0d0]
y = 2.0d0*x + 1.0d0

A = reshape([1.0d0, 3.0d0, &
             2.0d0, 4.0d0], [2,2])

b = matmul(A, x(1:2))
```

À retenir

Les deux langages permettent des opérations vectorisées et matricielles, avec des syntaxes et conventions d'indexation différentes.

Tableaux : Python et MATLAB

Python / NumPy

```
import numpy as np  
  
x = np.array([1, 2, 3])  
y = 2*x + 1
```

MATLAB

```
x = [1 2 3];  
y = 2*x + 1;
```

- MATLAB possède une syntaxe matricielle native.
- NumPy apporte les tableaux numériques à Python.
- L'indexation commence à zéro en Python et généralement à un en MATLAB.

Boucles : Python et MATLAB

Python

```
s = 0
for k in range(1, 6):
    s += k
```

MATLAB

```
s = 0;
for k = 1:5
    s = s + k; # indentation non obligatoire
end
```

Observation

Python utilise l'indentation ; MATLAB utilise le mot-clé end.

Problème : résoudre une équation

On cherche une racine de :

$$f(x) = 0.$$

Si f est continue sur $[a, b]$ et

$$f(a)f(b) < 0,$$

alors le théorème des valeurs intermédiaires garantit l'existence d'au moins une racine dans $[a, b]$.

Principe

Réduire progressivement l'intervalle contenant une racine.

Algorithme de bisection

À chaque étape :

- 1 calculer le milieu

$$m = \frac{a + b}{2};$$

- 2 évaluer le signe de $f(m)$;
- 3 conserver le sous-intervalle où le signe change ;
- 4 arrêter lorsque la précision demandée est atteinte.

Une condition d'arrêt possible :

$$\frac{b - a}{2} < \varepsilon.$$

Bisection en Python

```
def bisection(f, a, b, tol=1e-10, nmax=100):  
    fa = f(a)  
    fb = f(b)  
  
    if fa * fb > 0:  
        raise ValueError("Pas de changement de signe")  
  
    for k in range(nmax):  
        m = (a + b) / 2  
        fm = f(m)  
  
        if abs(fm) < tol or (b-a)/2 < tol:  
            return m, k+1  
  
        if fa * fm < 0:  
            b, fb = m, fm  
        else:  
            a, fa = m, fm  
  
    return (a+b)/2, nmax
```

Utilisation de la bisection

```
import numpy as np

def f(x):
    return x**3 - x - 2

racine, iterations = bisection(f, 1, 2)

print("Racine :", racine)
print("Iterations :", iterations)
print("Erreur :", abs(f(racine)))
```

Remarque

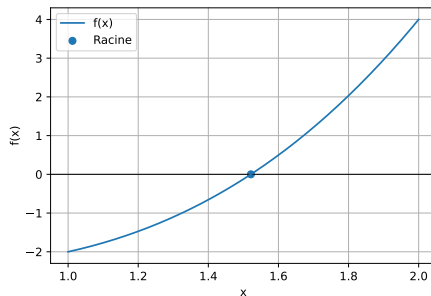
La méthode ne nécessite ni dérivée ni estimation initiale très précise.

Visualiser la recherche de racine

```
x = np.linspace(1, 2, 300)

plt.axhline(0, color="black", linewidth=0.8)
plt.plot(x, f(x), label="f(x)")
plt.scatter([racine], [f(racine)],
            label="Racine")

plt.xlabel("x")
plt.ylabel("f(x)")
plt.legend()
plt.grid(True)
plt.show()
```



Propriétés de convergence

Si l'intervalle initial a une longueur $L_0 = b - a$, alors après n itérations :

$$L_n = \frac{L_0}{2^n}.$$

Pour obtenir une longueur inférieure à ε :

$$n \geq \frac{\log(L_0/\varepsilon)}{\log 2}.$$

- Convergence garantie sous les hypothèses de continuité et de changement de signe.
- Convergence linéaire.
- Méthode robuste, mais parfois lente.

Bisection en MATLAB

```
function [m,k] = bisection(f,a,b,tol,nmax)
fa = f(a);

for k = 1:nmax
    m = (a+b)/2;
    fm = f(m);

    if abs(fm) < tol || (b-a)/2 < tol
        return
    end

    if fa*fm < 0
        b = m;
    else
        a = m;
        fa = fm;
    end
end
end
```

Bisection en Fortran

```
function f(x) result(y)
  implicit none
  real(8), intent(in) :: x
  real(8) :: y
  y = x**3 - x - 2.0d0
end function f
```

```
m = 0.5d0*(a+b)
fm = f(m)

if (fa*fm < 0.0d0) then
  b = m
else
  a = m
  fa = fm
end if
```

- Fortran impose généralement une déclaration explicite des variables.
- La précision `real(8)` est couramment utilisée pour les calculs en double précision.

- Utiliser des noms de variables explicites.
- Découper le programme en fonctions.
- Documenter les fonctions importantes.
- Vérifier les dimensions des tableaux.
- Tester les cas limites.
- Contrôler les tolérances numériques.
- Séparer calcul, affichage et visualisation.
- Utiliser un environnement virtuel lorsque le projet grandit.

Erreurs fréquentes

- Confondre liste Python et tableau NumPy.
- Utiliser `*` au lieu de `@` pour un produit matriciel.
- Oublier que l'indexation commence à zéro.
- Comparer des flottants avec une égalité stricte.
- Ne pas vérifier les hypothèses d'une méthode numérique.
- Créer des boucles Python inutiles alors qu'une opération NumPy vectorisée existe.

Exemple

Pour comparer deux réels, utiliser souvent une tolérance :

$$|x - y| < \varepsilon.$$

Exercice 1 : fonctions et tableaux

- 1 Créer un tableau de 100 points entre 0 et 2π .
- 2 Calculer $\sin(x)$ et $\cos(x)$.
- 3 Tracer les deux fonctions sur une même figure.
- 4 Ajouter une légende, un titre et une grille.

Exercice 2 : système linéaire

Résoudre :

$$\begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

- 1 Utiliser `np.linalg.solve`.
- 2 Vérifier le résidu $Ax - b$.
- 3 Comparer avec une résolution manuelle.

Exercice 3 : améliorer la bisection

Modifier la fonction de bisection pour :

- retourner l'historique des intervalles ;
- tracer la longueur de l'intervalle en fonction du nombre d'itérations ;
- comparer la précision obtenue pour plusieurs tolérances ;
- afficher un message si la méthode atteint `nmax`.

- Python fournit une syntaxe simple pour programmer.
- NumPy est la base du calcul scientifique vectorisé.
- Matplotlib permet de visualiser les résultats numériques.
- Les méthodes numériques se traduisent naturellement en fonctions.
- Python, MATLAB et Fortran peuvent implémenter les mêmes algorithmes.
- La fiabilité dépend des hypothèses mathématiques et des tests numériques.

Questions ?